

Sistema Web de Gestión de Tareas (CRUD)

Alumno: Francisco Vladimir Garcia Cardenas

Matricula: 1268662

Grupo: 1 **Año:** 4°

Fecha de entrega: 26/06/26

1. Nombre del proyecto

Task Manager API – Sistema de Gestión de Tareas

2. Objetivo

Desarrollar una aplicación web tipo API REST que permita la gestión de tareas mediante operaciones CRUD (Crear, Leer, Actualizar y Eliminar), utilizando ASP.NET Core Web API y una base de datos relacional con Entity Framework Core.

3. Tecnologías utilizadas

- ASP.NET Core Web API (.NET 10)
 - Entity Framework Core
 - SQL Server (LocalDB)
 - Swagger (OpenAPI)
 - Visual Studio 2026
 - GitHub (control de versiones)
-

4. Funcionalidades del sistema

El sistema permite:

- Crear tareas nuevas
 - Consultar todas las tareas
 - Consultar una tarea por ID
 - Actualizar tareas existentes
 - Eliminar tareas
-

5. Arquitectura del proyecto

El proyecto está estructurado en capas:

Controllers

Contiene los endpoints de la API.

Ejemplo:

- TareasController.cs
-

Models

Define las entidades del sistema.

Ejemplo:

```
public class Tarea
{
    public int Id { get; set; }
    public string Titulo { get; set; }
    public string Descripcion { get; set; }
    public bool Completada { get; set; }
}
```

Data

Contiene el contexto de base de datos.

```
public class AppDbContext : DbContext
{
    public DbSet<Tarea> Tareas { get; set; }
}
```

6. Base de datos

Se utilizó SQL Server LocalDB.

Nombre de la base de datos:

TaskManagerDB

Tabla principal:

Tareas

Estructura de la tabla:

Campo	Tipo	Descripción
Id	int	Identificador único
Titulo	string	Título de la tarea
Descripcion	string	Detalle de la tarea
Completada	bool	Estado de la tarea

7. Configuración del sistema

appsettings.json

```
{  
  "ConnectionStrings": {  
    "DefaultConnection":  
    "Server=(localdb)\\MSSQLLocalDB;Database=TaskManagerDB;Trusted_Connection=True;TrustServerCertificate=True;"  
  }  
}
```

Program.cs

```
builder.Services.AddControllers();  
  
builder.Services.AddDbContext<AppDbContext>(options =>  
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));  
  
builder.Services.AddEndpointsApiExplorer();  
builder.Services.AddSwaggerGen();
```

8. Endpoints de la API

- ◆ **Obtener todas las tareas**

GET /api/Tareas

- ◆ **Obtener tarea por ID**

GET /api/Tareas/{id}

- ◆ **Crear tarea**

POST /api/Tareas

- ♦ **Actualizar tarea**

PUT /api/Tareas/{id}

- ♦ **Eliminar tarea**

DELETE /api/Tareas/{id}

9. Evidencia de funcionamiento

La API se probó utilizando Swagger UI, permitiendo ejecutar cada operación CRUD directamente desde el navegador.

Ejemplo:

- Creación de tarea exitosa (201 Created)
 - Consulta de tareas (200 OK)
 - Actualización de tarea (200 OK)
 - Eliminación de tarea (204 No Content)
-

10. Control de versiones (GitHub)

El proyecto fue gestionado utilizando GitHub para control de versiones.

Repositorio:

<https://github.com/fvgarcia93/gestion-tareas-crud>

11. Conclusión

El proyecto permite comprender la implementación de una API REST utilizando ASP.NET Core y Entity Framework, integrando una base de datos relacional y aplicando operaciones CRUD fundamentales para sistemas de gestión de información.